DATA ANALYST INTERNSHIP

Task 4: SQL for Data Analysis

Submitted by: Mohit Kumar

Tool Used: MySQL Community Server 8.0 + MySQL Workbench

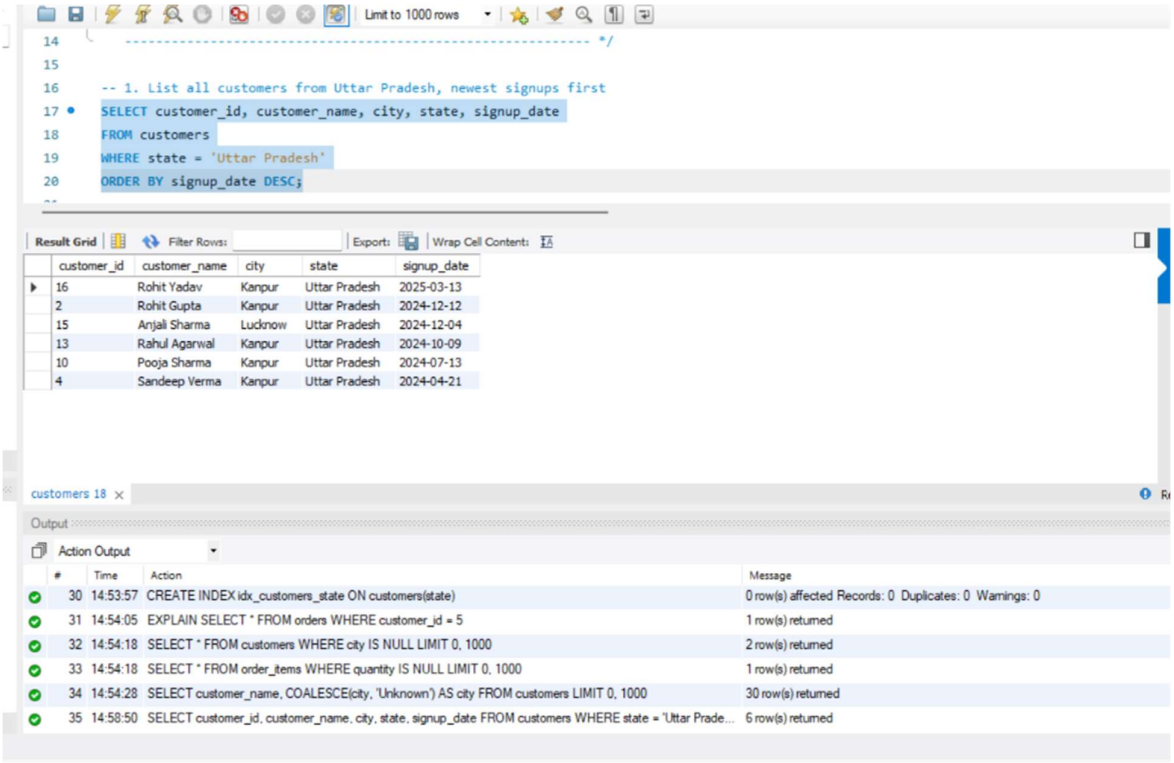
Database: ecommerce_db (Customers, Products, Orders, Order Items)

Objective	Use SQL queries to extract and analyze data from a database.
Tools	MySQL Community Server, MySQL Workbench
Deliverables	SQL queries in a SQL file + screenshots of output
Dataset	Custom-built Ecommerce database (customers, products, orders, order_items)
Outcome	Learn to manipulate and query structured data using SQL

a. SELECT, WHERE, ORDER BY, GROUP BY

Query 1: List all customers from Uttar Pradesh, newest signups first

```
SELECT customer_id, customer_name, city, state, signup_date
FROM customers
WHERE state = 'Uttar Pradesh'
ORDER BY signup_date DESC;
```



Screenshot: Output of Query 1 in MySQL Workbench

Query 2: List all products priced above 1000, cheapest first

```
SELECT product_id, product_name, category, price
FROM products
WHERE price > 1000
ORDER BY price ASC;
```

```
20      ORDER BY signup_date DESC;
21
22      -- 2. List all products priced above 1000, cheapest first
23      SELECT product_id, product_name, category, price
24      FROM products
25      WHERE price > 1000
26      ORDER BY price ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	category	price
2	Bluetooth Headphones	Electronics	1299.00
8	Backpack	Accessories	1599.00
11	Running Shoes	Fashion	1999.00
10	Smartwatch	Electronics	2499.00
5	Study Table	Furniture	3499.00
4	Office Chair	Furniture	4999.00

Screenshot: Output of Query 2 in MySQL Workbench

Query 3: Count how many orders exist per status

```
SELECT status, COUNT(*) AS total_orders
FROM orders
GROUP BY status
ORDER BY total_orders DESC;
```

Limit to 1000 rows

```
26      ORDER BY price ASC;
27
28      -- 3. Count how many orders exist per status
29      SELECT status, COUNT(*) AS total_orders
30      FROM orders
31      GROUP BY status
32      ORDER BY total_orders DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

status	total_orders
Delivered	48
Cancelled	21
Shipped	17
Pending	14

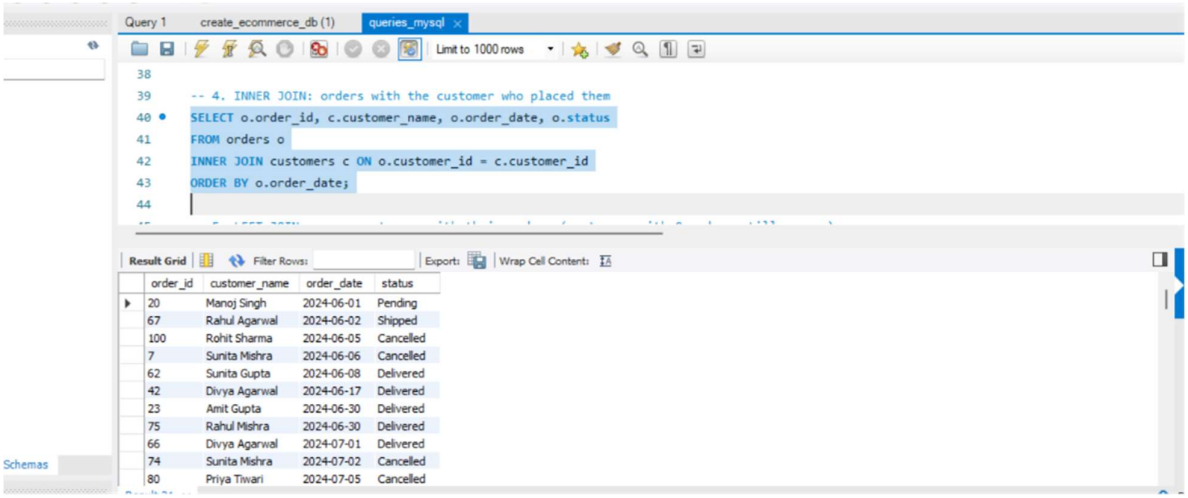
Result Grid | Form Editor | Field Types | Result 20 | Read Only | Output

Screenshot: Output of Query 3 in MySQL Workbench

b. JOINS (INNER, LEFT, RIGHT)

Query 4: INNER JOIN — orders with the customer who placed them

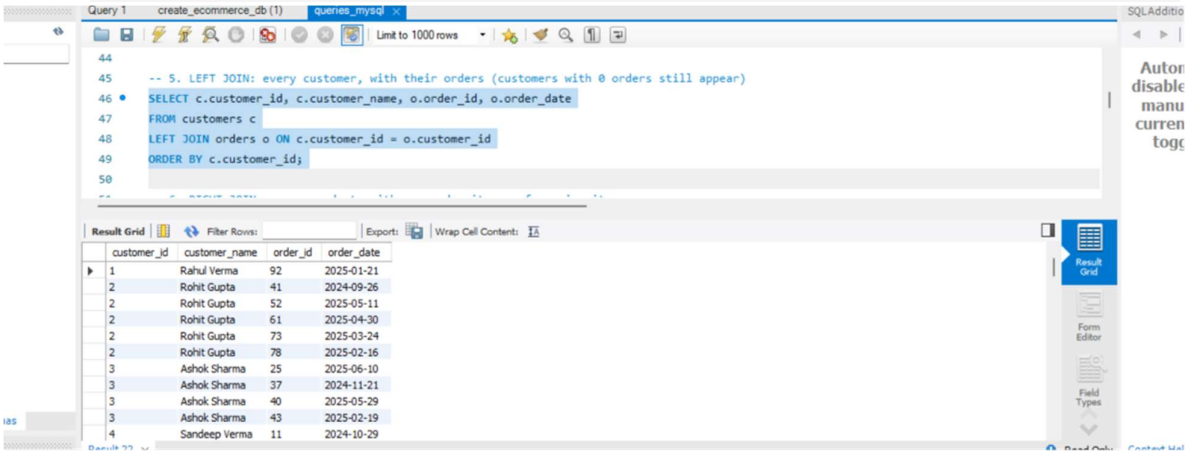
```
SELECT o.order_id, c.customer_name, o.order_date, o.status
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id
ORDER BY o.order_date;
```



Screenshot: Output of Query 4 in MySQL Workbench

Query 5: LEFT JOIN — every customer, with their orders (customers with 0 orders still appear)

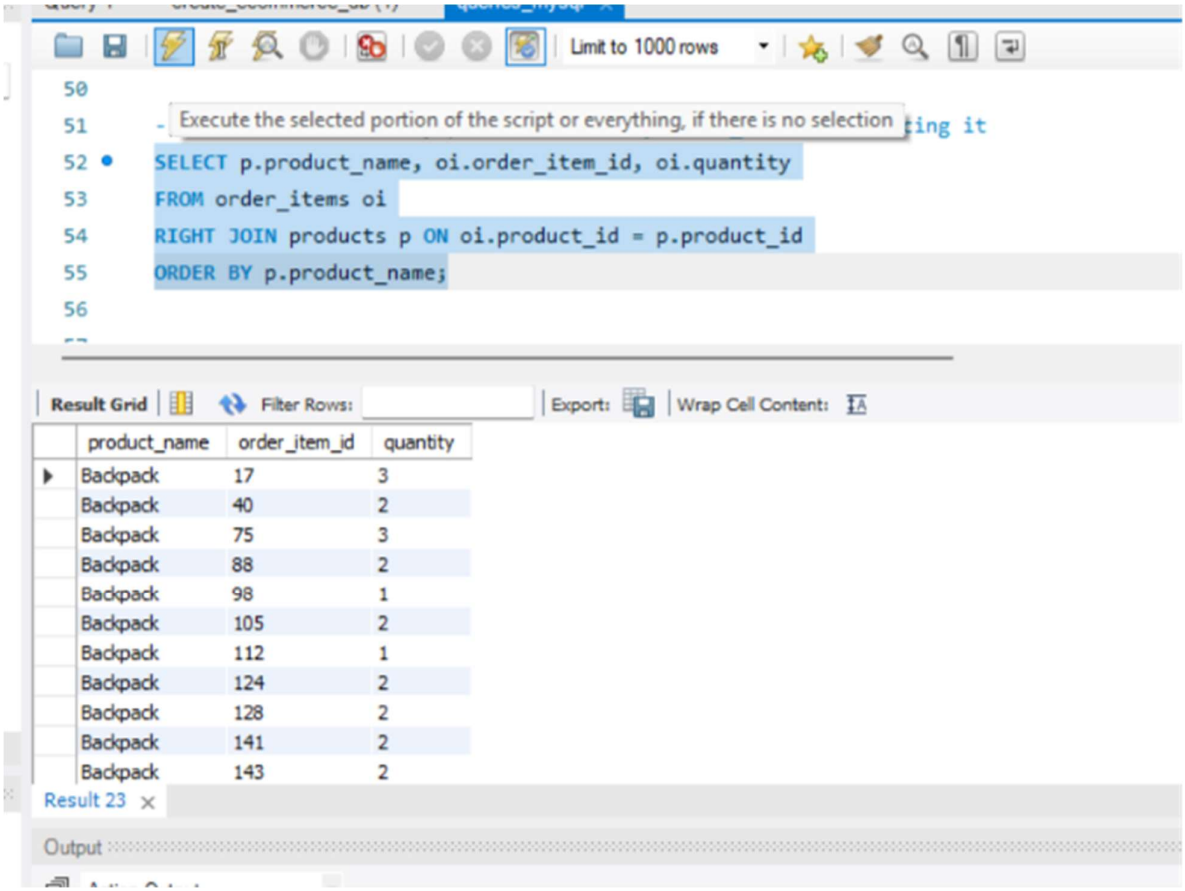
```
SELECT c.customer_id, c.customer_name, o.order_id, o.order_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
ORDER BY c.customer_id;
```



Screenshot: Output of Query 5 in MySQL Workbench

Query 6: RIGHT JOIN — every product, with any order_items referencing it

```
SELECT p.product_name, oi.order_item_id, oi.quantity
FROM order_items oi
RIGHT JOIN products p ON oi.product_id = p.product_id
ORDER BY p.product_name;
```

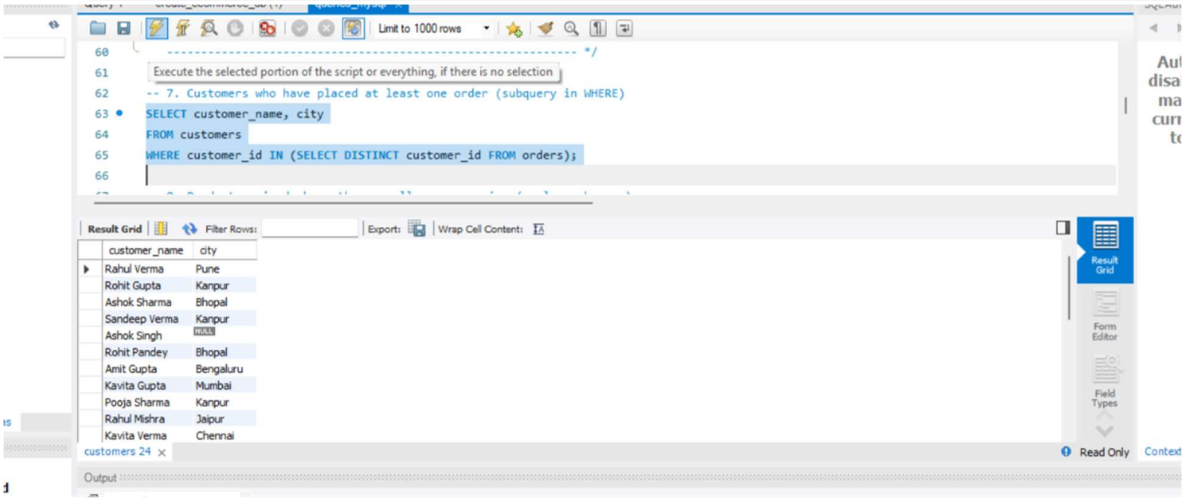


Screenshot: Output of Query 6 in MySQL Workbench

c. Subqueries

Query 7: Customers who have placed at least one order (subquery in WHERE)

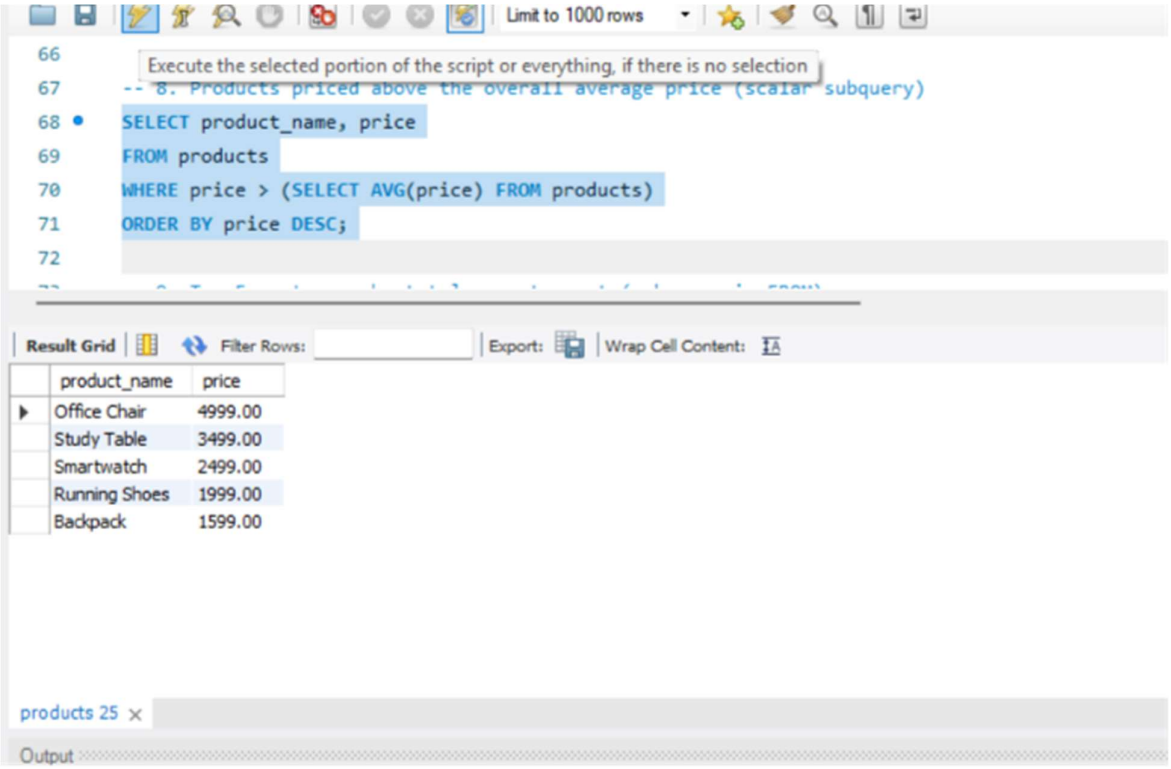
```
SELECT customer_name, city
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```



Screenshot: Output of Query 7 in MySQL Workbench

Query 8: Products priced above the overall average price (scalar subquery)

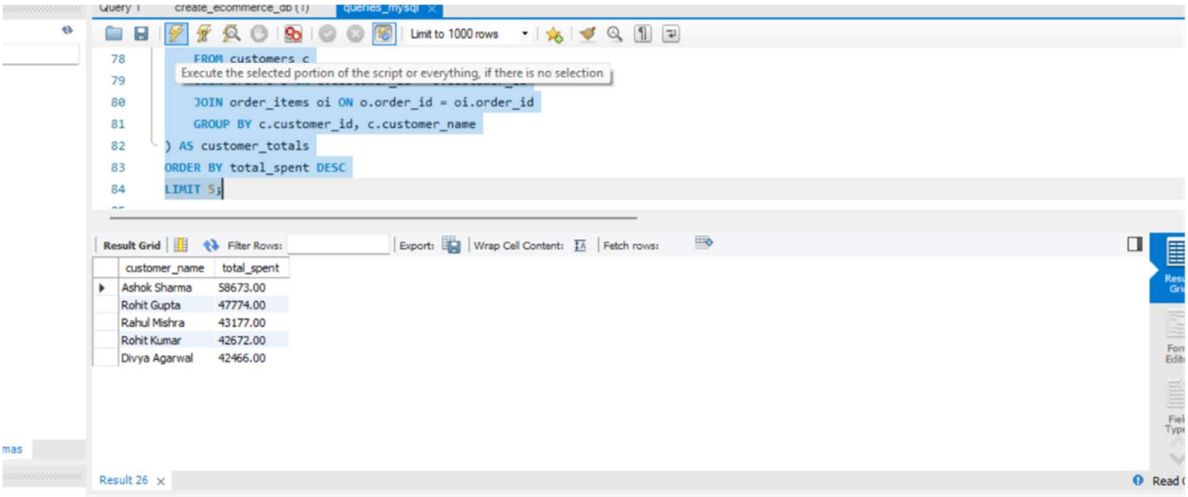
```
SELECT product_name, price
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```



Screenshot: Output of Query 8 in MySQL Workbench

Query 9: Top 5 customers by total amount spent (subquery in FROM)

```
SELECT customer_name, total_spent
FROM (
SELECT c.customer_name, SUM(oi.quantity * oi.unit_price) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY c.customer_id, c.customer_name
) AS customer_totals
ORDER BY total_spent DESC
LIMIT 5;
```

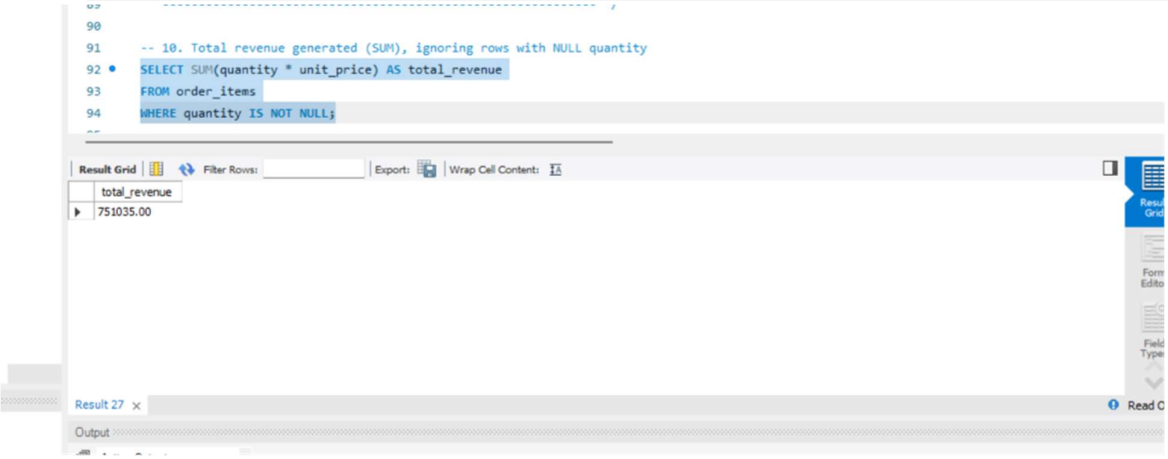


Screenshot: Output of Query 9 in MySQL Workbench

d. Aggregate Functions (SUM, AVG)

Query 10: Total revenue generated (SUM), ignoring rows with NULL quantity

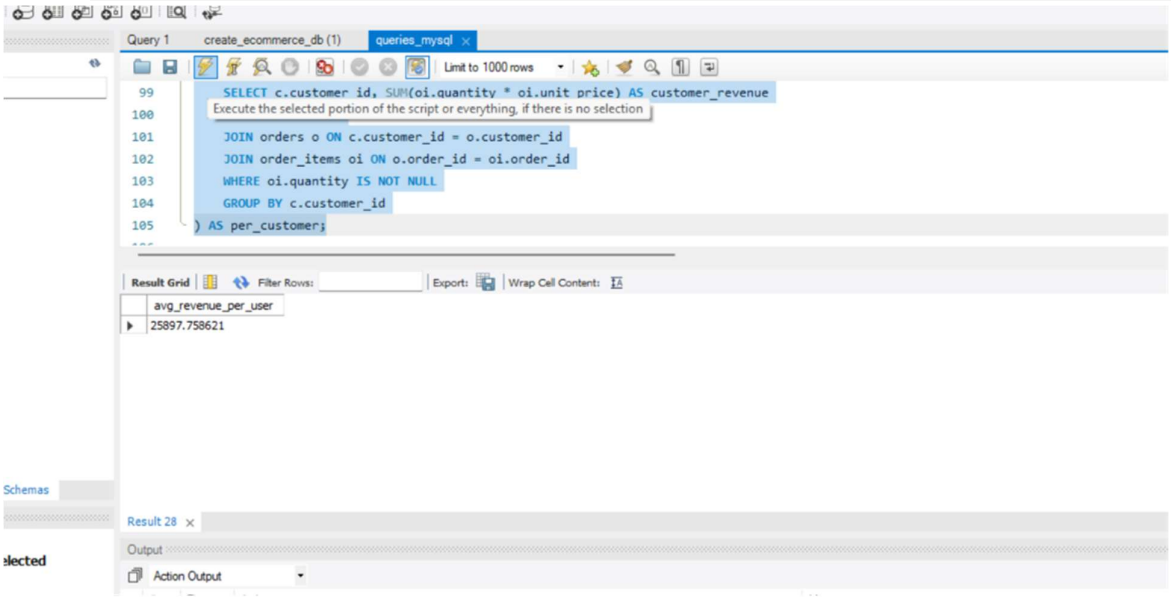
```
SELECT SUM(quantity * unit_price) AS total_revenue
FROM order_items
WHERE quantity IS NOT NULL;
```



Screenshot: Output of Query 10 in MySQL Workbench

Query 11: Average revenue per user (SUM per customer, then AVG across customers)

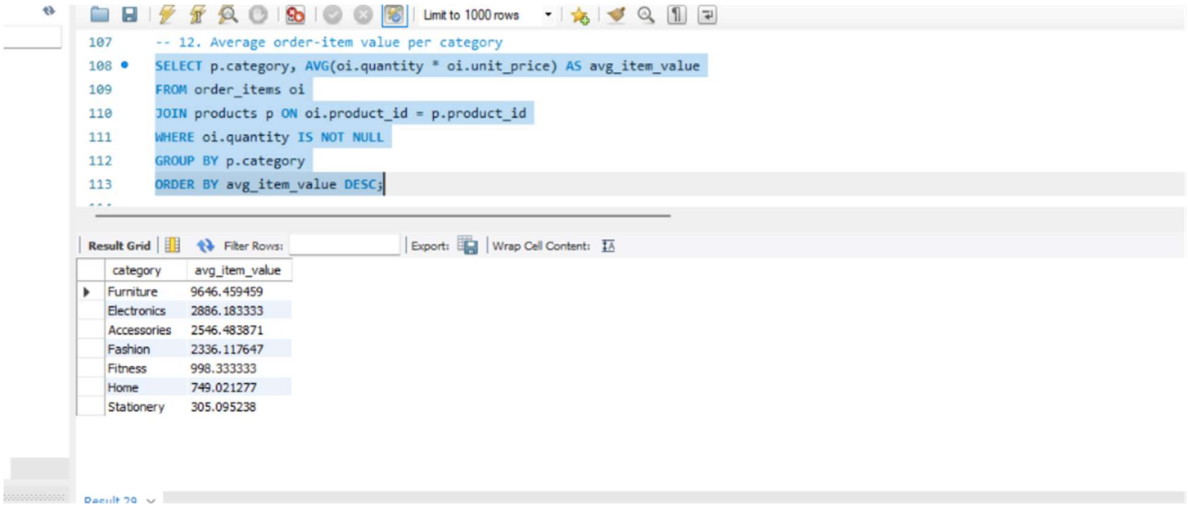
```
SELECT AVG(customer_revenue) AS avg_revenue_per_user
FROM (
SELECT c.customer_id, SUM(oi.quantity * oi.unit_price) AS customer_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.quantity IS NOT NULL
GROUP BY c.customer_id
) AS per_customer;
```



Screenshot: Output of Query 11 in MySQL Workbench

Query 12: Average order-item value per category

```
SELECT p.category, AVG(oi.quantity * oi.unit_price) AS avg_item_value
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
WHERE oi.quantity IS NOT NULL
GROUP BY p.category
ORDER BY avg_item_value DESC;
```



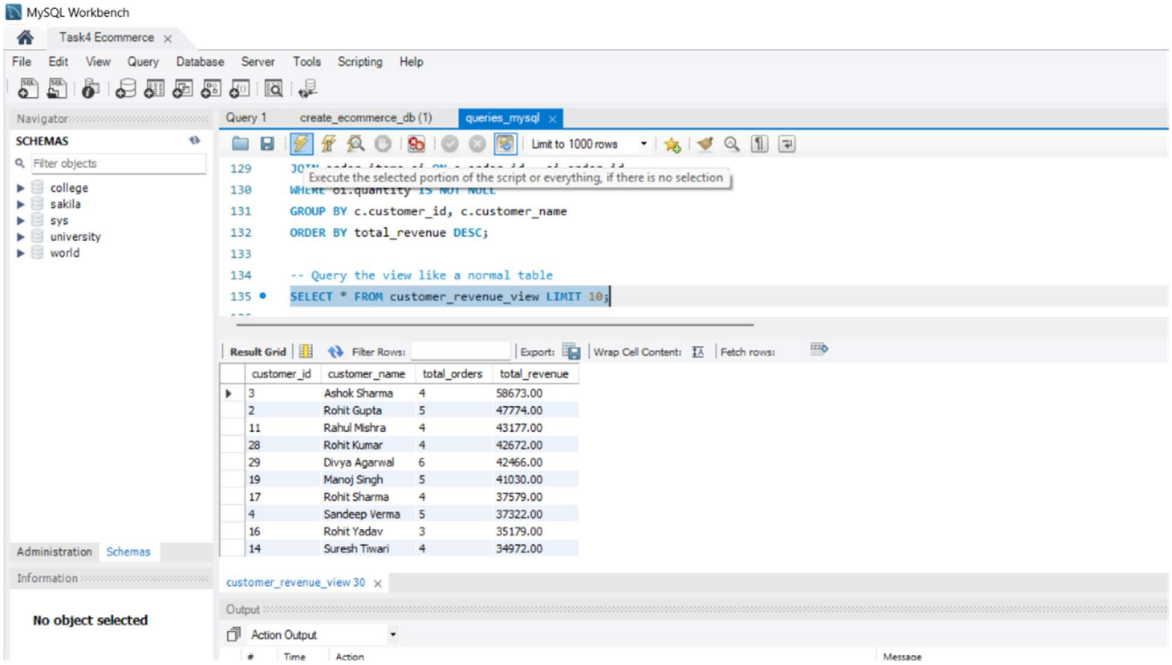
Screenshot: Output of Query 12 in MySQL Workbench

e. Create Views for Analysis

View: customer_revenue_view

```
CREATE VIEW customer_revenue_view AS
SELECT c.customer_id,
c.customer_name,
COUNT(DISTINCT o.order_id) AS total_orders,
SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.quantity IS NOT NULL
GROUP BY c.customer_id, c.customer_name
ORDER BY total_revenue DESC;

-- Query the view like a normal table
SELECT * FROM customer_revenue_view LIMIT 10;
```



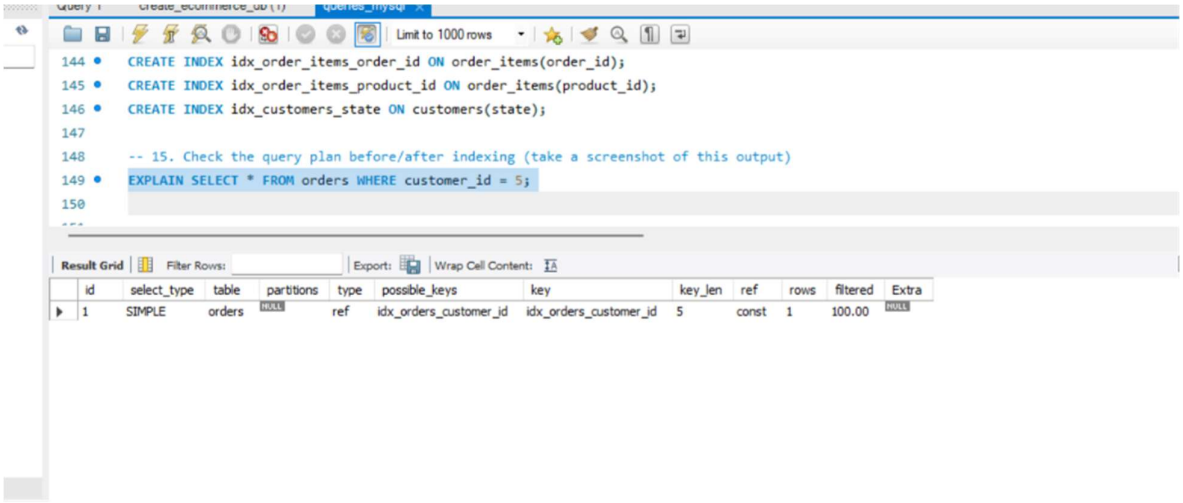
Screenshot: customer_revenue_view created and queried successfully

f. Optimize Queries with Indexes

Creating indexes + checking the query plan

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);
CREATE INDEX idx_customers_state ON customers(state);

-- Check the query plan after indexing
EXPLAIN SELECT * FROM orders WHERE customer_id = 5;
```

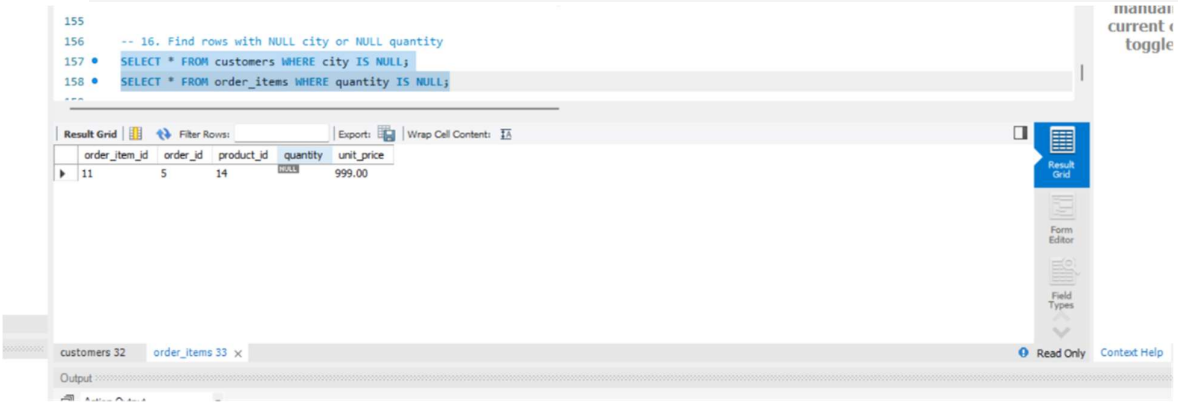


Screenshot: Indexes created; EXPLAIN shows MySQL now uses idx_orders_customer_id (type = ref) instead of scanning the full orders table

Bonus. Handling NULL Values

Query 16: Find rows with NULL city or NULL quantity

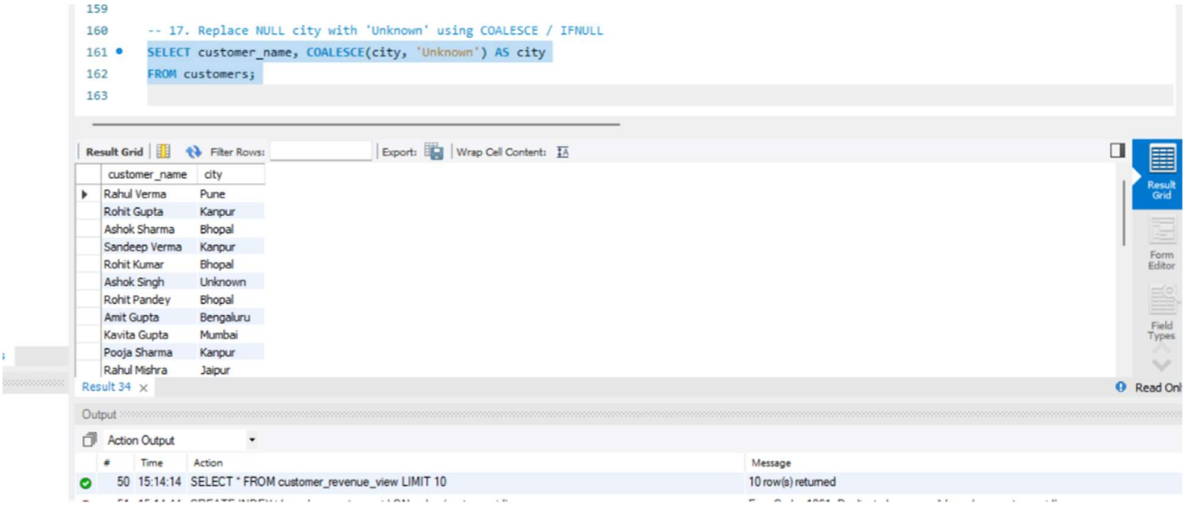
```
SELECT * FROM customers WHERE city IS NULL;
SELECT * FROM order_items WHERE quantity IS NULL;
```



Screenshot: Output of Query 16 in MySQL Workbench

Query 17: Replace NULL city with 'Unknown' using COALESCE

```
SELECT customer_name, COALESCE(city, 'Unknown') AS city
FROM customers;
```



Screenshot: Output of Query 17 in MySQL Workbench

Interview Questions & Answers

1. What is the difference between WHERE and HAVING?

WHERE filters individual rows before any grouping happens, and it cannot use aggregate functions like SUM() or COUNT(). HAVING filters groups after GROUP BY has been applied, and it is specifically used to filter on aggregate results, e.g. HAVING SUM(quantity) > 10.

2. What are the different types of joins?

INNER JOIN returns only matching rows from both tables. LEFT JOIN returns all rows from the left table plus matches from the right (unmatched right columns are NULL). RIGHT JOIN does the reverse — all rows from the right table plus matches from the left. FULL OUTER JOIN (not directly supported in MySQL, simulated with UNION of LEFT and RIGHT JOIN) returns all rows from both sides. A CROSS JOIN returns every combination of rows from both tables.

3. How do you calculate average revenue per user in SQL?

First calculate each customer's total revenue by joining orders and order_items and using SUM(quantity * unit_price) grouped by customer_id. Then wrap that result in an outer query and apply AVG() across all the per-customer totals, as shown in Query 11.

4. What are subqueries?

A subquery is a SELECT statement nested inside another query. It can appear in the WHERE clause (to filter based on another result set), in the FROM clause (as a temporary derived table), or as a scalar value in the SELECT list. Subqueries let you break a complex problem into smaller, more readable steps, as shown in Queries 7–9.

5. How do you optimize a SQL query?

Common techniques include: creating indexes on columns used in JOIN, WHERE, and ORDER BY clauses; avoiding SELECT * and only selecting needed columns; using EXPLAIN to see the query execution plan and check whether indexes are actually being used; avoiding unnecessary subqueries when a JOIN would do the same job faster; and filtering data as early as possible in the query.

6. What is a view in SQL?

A view is a saved, virtual table defined by a SELECT query. It doesn't store data itself — it runs the underlying query each time it's accessed. Views are useful for simplifying repeated complex queries (like customer_revenue_view in this task) and for controlling what data users can see.

7. How would you handle null values in SQL?

Use IS NULL / IS NOT NULL to detect or filter them (since NULL can't be compared with =). Use COALESCE() or IFNULL() to substitute a default value when a column is NULL, as shown in Query 17. When doing aggregate calculations like SUM or AVG, remember that most aggregate functions automatically ignore NULLs, but it's still good practice to filter them explicitly when they would distort a calculation (e.g. WHERE quantity IS NOT NULL in this task).